

Project Report: Healthcare Risk Prediction Model & Web Application

1. Project Overview

This document serves as a comprehensive report detailing the end-to-end development of the Healthcare Risk Prediction machine learning project. The primary objective of this project was to build a predictive data pipeline that estimates the risk of patient complications based on hospital admission data, and to subsequently deploy this model as an interactive, public-facing web application.

This project bridged several core data science competencies: data cleaning and feature engineering in Jupyter Notebooks, training a classification algorithm, exporting the model, and finally transitioning from a local environment to a cloud-hosted web interface.

2. Technology Stack & Tools Used

The project leveraged a modern Python data science and deployment stack to ensure efficiency and scalability:

Category	Tool / Library	Specific Purpose in Project
Data Manipulation	Pandas	Merging datasets, extracting integer days from timedelta objects, and structuring the prediction DataFrame.
Machine Learning	Scikit-Learn	Train-test splitting, Logistic Regression algorithm implementation, and calculating ROC/AUC metrics.
Data Visualization	Matplotlib	Plotting the Receiver Operating Characteristic (ROC) curve to visualize model accuracy.
Model Serialization	Joblib	Exporting the trained model from system RAM into a persistent .pkl file.

Category	Tool / Library	Specific Purpose in Project
Web Development	Streamlit	Building the interactive user interface (UI) to collect inputs and display the prediction output.
Deployment & Hosting	GitHub & Streamlit Community Cloud	Version control (storing the code online) and cloud hosting for the live web application.

3. Step-by-Step Implementation Guide

Step 1: Data Processing & Feature Engineering (Jupyter Notebook)

- **Data Integration:** The core patients dataset was merged with external diagnoses and outcomes datasets utilizing shared primary keys (e.g., OutcomeID, DiagnosisID).
- **Datetime Conversion & Calculation:** AdmissionDate and DischargeDate columns were converted to proper Pandas datetime objects. A new feature representing the length of the hospital stay was engineered by subtracting the admission date from the discharge date.
- **DType Resolution:** To ensure compatibility with mathematical algorithms, the resulting timedelta object for the length of stay was stripped down to raw integers using the .dt.days accessor.
- **Target Encoding:** The categorical OutcomeName string was mapped into a machine-readable binary numerical format (outcomeEncoded) using a dictionary mapping: 'Recovered' to 0, and 'Complicated' or 'Deceased' to 1.

Step 2: Model Training

- **Feature Selection:** Isolated the predictive independent variables: Age, lenghofsay (Length of Stay), TreatmentCost, and AbnormalLabcount.
- **Data Splitting:** Utilized the train_test_split module to divide the data into a training set (70%) for the model to learn from, and a testing set (30%) to evaluate unseen data.
- **Algorithm Initialization:** Instantiated a LogisticRegression model (with max_iter=500 to ensure convergence) and fitted it to the training data.

Step 3: Model Evaluation & Visualization

- **Probability Extraction:** Used the `.predict_proba()` method on the test set to extract the likelihood array for the positive risk class.
- **ROC/AUC Plotting:** Calculated the False Positive Rate (FPR), True Positive Rate (TPR), and Area Under the Curve (AUC). Plotted a customized ROC graph utilizing `matplotlib.pyplot`, complete with diagonal baselines and styled legends, to validate the model's diagnostic threshold.

Step 4: Model Export (Serialization)

- **Creating the Artifact:** Executed `joblib.dump(model, 'Risk_model.pkl')` within a new code cell. This critical step serialized the trained algorithm, saving it from temporary memory as a permanent `.pkl` file on the local hard drive, enabling it to be transferred to the web application.

Step 5: Web Application Architecture

- **Environment Shift:** Transitioned out of the Jupyter Notebook environment and authored a standard Python script named `app.py` in VS Code.
- **UI Design:** Utilized Streamlit functions (`st.title`, `st.number_input`) to create a clean user interface that captures Patient Age, Length of Stay, and Treatment Cost.
- **Prediction Logic execution:** Engineered a "Predict" button (`st.button`) that constructs a Pandas DataFrame on the fly from the user inputs. This DataFrame is passed into the loaded `Risk_model.pkl`, returning both a definitive classification ("High Risk" or "Low Risk") and an exact probability percentage back to the user.

Step 6: Cloud Deployment

- **Dependency Management:** Authored a `requirements.txt` file listing the exact libraries (`streamlit`, `pandas`, `joblib`, `scikit-learn`) required for the cloud container to successfully build the environment.
- **Version Control:** Uploaded the three essential application files (`app.py`, `Risk_model.pkl`, and `requirements.txt`) to a centralized public GitHub repository.
- **Live Hosting:** Linked the GitHub repository to the Streamlit Community Cloud interface. Configured a custom subdomain (e.g., bypassing the automatic file-path URL error) to successfully generate a live, shareable URL for the application.